Дисциплина: Основы обработки цифровых изображений
Студент: Онищук Тимофей Дмитриевич
Группа: 3 курс, 2 группа
Вариант: 6 (Глобальная пороговая обработка, контрастирование)

Отчёт по лабораторной работе №2
Тема: Алгоритмы обработки изображений: пороговая бинаризация и поэлементные преобразования

1. Цель работы

Изучение базовых методов цифровой обработки изображений, реализация алгоритмов глобальной пороговой бинаризации (ручной метод и метод Оцу), а также поэлементных операций (негатив, осветление) и линейного контрастирования. Разработка оконного приложения с графическим интерфейсом для применения изученных методов к загружаемым изображениям и оценки качества их работы.

2. Теоретические сведения и формулы преобразования

2.1. Глобальная пороговая обработка

Глобальная бинаризация заключается в разделении пикселей изображения на два класса (черный и белый) по заданному порогу яркости T. Пусть $f(x, y)$ — исходная яркость пикселя, а $g(x, y)$ — результирующая. Тогда:

$$g(x,y) = \begin{cases} 255, & \text{если } f(x,y) > T \\ 0, & \text{если } f(x,y) \leq T \end{cases}$$

Метод Оцу (Otsu) позволяет вычислить оптимальный порог T автоматически. Алгоритм ищет такое значение $T \in [0, 255]$, которое минимизирует внутриклассовую дисперсию $\sigma_w^2(T)$ (или максимизирует межклассовую). Внутриклассовая дисперсия определяется как:

$$\sigma_w^2(T) = \omega_0(T)\sigma_0^2(T) + \omega_1(T)\sigma_1^2(T)$$

где ω_0, ω_1 — вероятности попадания пикселей в классы (ниже и выше порога), а σ_0^2, σ_1^2 — дисперсии яркостей внутри этих классов.

2.2. Линейное контрастирование и поэлементные операции

Линейное контрастирование применяется для расширения динамического диапазона изображения. Операция «растягивает» гистограмму так, чтобы минимальная яркость f_{min} стала 0, а максимальная f_{max} стала 255:

$$g(x, y) = \frac{f(x, y) - f_{min}}{f_{max} - f_{min}} \times 255$$

К поэлементным операциям относятся преобразования каждого пикселя независимо от его соседей. Например, операция «Негатив» (инверсия): $g(x, y) = 255 - f(x, y)$. Операция изменения яркости (сдвиг): $g(x, y) = \min(f(x, y) + C, 255)$, где C — константа.

3. Описание интерфейса и тестирования

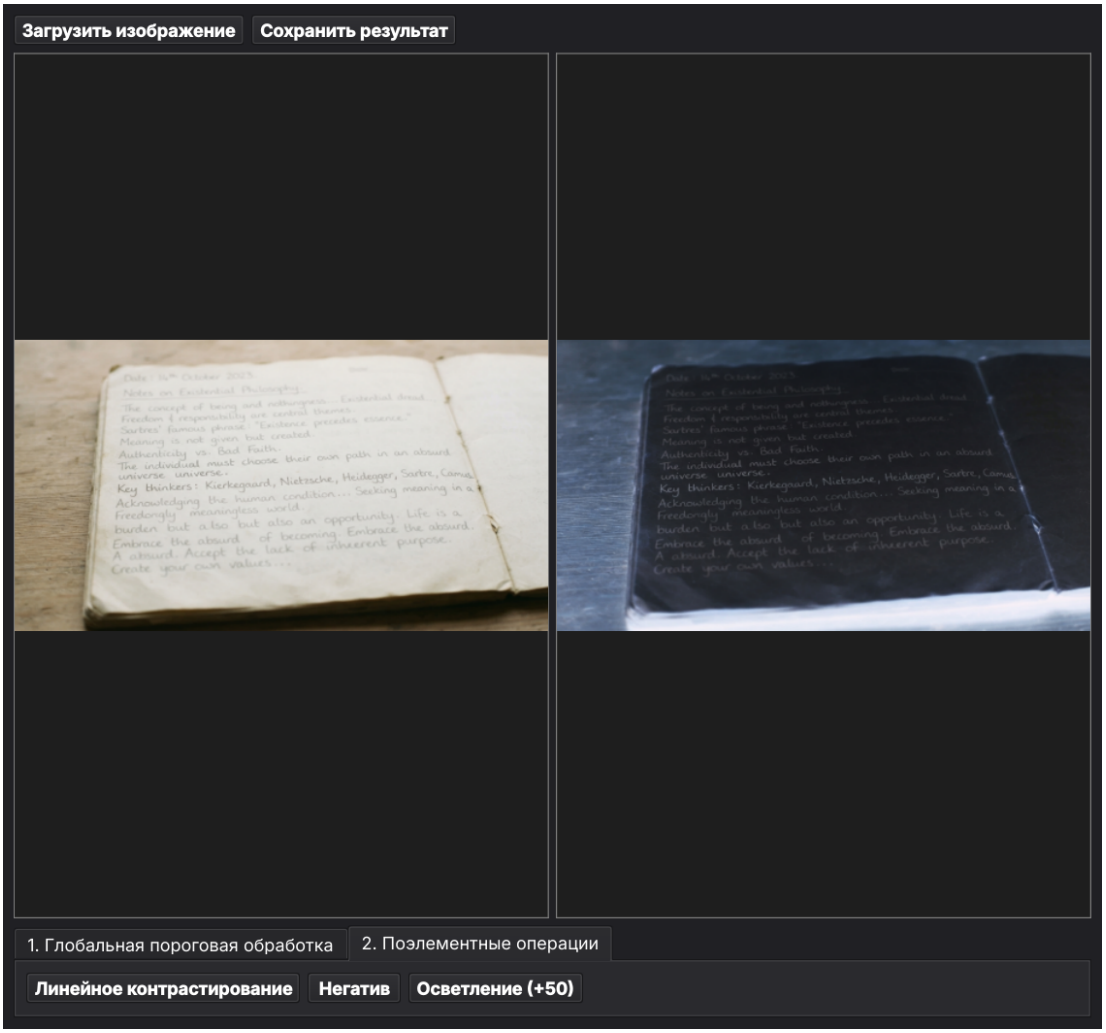
Приложение разработано на языке Python с применением фреймворка `PyQt6` для создания нативного графического интерфейса под дисплейные серверы `Wayland/X11` и библиотеки компьютерного зрения `OpenCV` для математической обработки матриц. Интерфейс разделен на:

1. **Панель ввода/вывода**: кнопки загрузки исходного изображения и сохранения результата на диск.
2. **Область визуализации**: синхронное отображение исходного («Оригинал») и обработанного («Результат») изображений с автоматическим масштабированием пропорций.
3. **Панель инструментов (вкладки)**:
   - *Глобальная пороговая обработка*: ползунок для ручного подбора порога $T \in [0, 255]$ и кнопка авто-бинаризации методом Оцу.
   - *Поэлементные операции*: кнопки вызова линейного контрастирования, создания негатива и добавления константы к яркости (+50).

При тестировании на затемненных снимках документов метод Оцу показал превосходство над ручным подбором: алгоритм автоматически адап-

тируется под освещенность листа, не допуская заливки полезного текста сплошным черным или белым цветом.

4. Скриншот работающего приложения



5. Ключевые фрагменты программного кода

```
# 1.
                                                (val)
def apply_manual(self):
    if self.original_image is None: return
    gray = cv2.cvtColor(self.original_image, cv2.COLOR_BGR2GRAY)
    val = self.slider.value()
    _, self.processed_image = cv2.threshold(gray, val, 255, cv2.
        THRESH_BINARY)
    self.update_images()

# 2.

def apply_otsu(self):
    if self.original_image is None: return
    gray = cv2.cvtColor(self.original_image, cv2.COLOR_BGR2GRAY)
    #                   0                                          ,
        OpenCV
```

```
    val, self.processed_image = cv2.threshold(gray, 0, 255,
                                              cv2.THRESH_BINARY + cv2.
                                                  THRESH_OTSU)
    self.update_images()

# 3.                                          :
                                      (                          MinMax)
def linear_contrast(self):
    if self.original_image is None: return
    self.processed_image = cv2.normalize(self.original_image, None,
                                         0, 255, cv2.NORM_MINMAX)
    self.update_images()

# 4.                                          :                 (
                    )
def negative(self):
    if self.original_image is None: return
    self.processed_image = cv2.bitwise_not(self.original_image)
    self.update_images()
```

Листинг 1: Реализация алгоритмов обработки изображений (PyQt6 + OpenCV)

6. Вывод

В ходе выполнения лабораторной работы были изучены и практически реализованы базовые алгоритмы фильтрации и цветокоррекции изображений. Разработанное приложение на базе `PyQt6` и `OpenCV` успешно справляется с возложенными задачами. Сравнение методов показало, что адаптивный математический подход (метод Оцу) в большинстве случаев превосходит ручное задание порога при бинаризации текстов. Линейное контрастирование подтвердило свою эффективность при восстановлении потерянного динамического диапазона (выцветших и тусклых фотографий).